

How I Built the Satellite Orbit Simulation — In My Own Words

Where I Started

When I first read the project requirements, the idea of simulating a satellite orbiting Earth felt intimidating. I didn't know where to begin. So I started simple — I asked myself: "*What does a satellite orbit actually look like?*" It's just a circular path around Earth, with a small object moving along it. That mental picture is what I built toward.

I chose **JavaScript with the HTML5 Canvas API** because it runs directly in the browser — no installations, no setup headaches. Anyone can open the index.html file and it just works. That felt right for a project meant to be visual and interactive.

How I Structured It

I split the project into **three files**, each with a clear responsibility:

- **index.html** — The skeleton. It defines the layout: the canvas area on the left where the simulation runs, and the control panel on the right where you adjust the satellites.
 - **style.css** — The look and feel. I went with a dark space theme — deep navy backgrounds, glowing cyan and orange accents, and a monospace font that gives it that "mission control" feeling.
 - **simulation.js** — The brain. All the physics, animation, and interactivity live here.
-

The Physics — Kepler's Third Law

This was the most interesting part. I didn't want the satellite to just spin randomly — I wanted it to move based on **real orbital mechanics**.

The formula I used is **Kepler's Third Law**:

$$T = 2\pi \times \sqrt{(a^3 / \mu)}$$

- **T** is the orbital period — how long one full orbit takes
- **a** is the orbital radius — the distance from Earth's center to the satellite
- **μ** is Earth's gravitational parameter — a fixed constant: **398,600 km³/s²**

What this means in plain English: **the farther a satellite is from Earth, the longer it takes to complete one orbit**. You can actually see this in the simulation — if you drag Satellite Beta's orbit radius outward, it visibly slows down compared to Alpha. That's real physics playing out on screen.

I also calculated **orbital velocity** using:

$$v = \sqrt{(\mu / a)}$$

This tells you how fast the satellite is actually moving in km/s — and this number updates live on the panel as you adjust the sliders.

How the Animation Works

The animation runs using `requestAnimationFrame()` — the browser's built-in loop that redraws the canvas roughly 60 times per second. Every frame, I:

1. Calculate how much time has passed (dt — delta time in seconds)
2. Move each satellite forward along its orbit by angle $+= \omega \times dt$ (where ω is angular velocity)
3. Record the satellite's position into a **trail array** so the glowing tail effect appears
4. Clear the canvas and redraw everything — stars, Earth, orbit paths, trails, satellites

The Earth itself is drawn entirely with Canvas shapes — gradients for the ocean, green ellipses clipped into a circle for continents, and a radial gradient for the atmosphere glow. No images were used at all.

The Controls

Each satellite has two sliders:

- **Orbit Radius** — changes how far the satellite orbits from Earth (maps to real altitudes: 300 km to 2,000 km)
- **Speed** — multiplies the angular velocity so you can watch the physics at different paces

When you move a slider, the orbital period, altitude, and velocity displayed on the panel all update instantly using the physics formulas. The simulation runs at **1000× real-time** by default — otherwise a real 90-minute orbit would take... 90 minutes.

What I Learned

- How to use the **Canvas API** to draw and animate 2D graphics entirely in code
- How **Kepler's laws** translate into actual numbers and motion you can see
- How **delta-time animation** makes movement smooth regardless of frame rate
- How to structure a frontend project cleanly across HTML, CSS, and JS files
- How to connect physics formulas to real interactive UI controls

Tech Stack Summary

Technology	Why I Used It
HTML5 Canvas	Render the simulation visually in real-time
CSS3	Dark space-themed layout and glowing effects
Vanilla JavaScript	Physics engine, animation loop, event handling
Kepler's Third Law	Real orbital mechanics for period & velocity
Google Fonts (Orbitron)	Mission-control aesthetic for the UI
VS Code + Live Server	Development environment
GitHub	Version control and project submission
